

NGR Close Train Operating System

Role Profile System — Foundational Document v1.4

Updated May 2026 — incorporates field testing through firmware v2.0

Document History

v1.0 — Initial foundational document. Defined four stop profiles (DEFAULT, STATION, TRAILING, LEADER), role-based parameter system, and runtime adjustability via Dispatcher.

v1.1 — Safety corrections from field review. Added confirmed-stop debounce requirement for RELEASE, corrected dwell sequencing, gated leader departure on both dwell-expired AND RELEASE-received, added phase-lock prevention, and mandated decisive trailing stop.

v1.2 — Platform behavior introduced. Added MSG_TYPE_DEPARTING, trailing platform advance sequence (five sub-states), unified ramp rate, and differentiated leader/trailing ramp delays for positional control. Fixed deadlock caused by DEPARTING not being sent on STATION departures.

v1.3 — Full runtime tunability. Introduced rampUpRateMs as a named runtime-adjustable variable, advance PWM override for trailing platform approach, refined default timing values from field testing, and complete inline annotation of both loco and Dispatcher sketches.

v1.4 — Signal-gated platform release. TR_PLATFORM_DWELL now requires a MSG_TYPE_BLOCK from the leader at a block different from the encounter block before the trailing loco departs. The dwell timer is retained as a minimum floor but is no longer the sole release condition. This closes a field-confirmed failure mode where timing-based release caused the trailing loco to depart the platform before the leader reached the next station. Establishes the foundational principle: release conditions must be signal-based, not timer-based. Firmware baseline: v2.0 production build.

Core Principles

Role-based behavior. A locomotive's stopping and departure behavior is determined entirely by its role at the moment it stops. Role is not fixed to a locomotive — it is assigned dynamically based on the locomotive's relationship to other traffic at each block boundary. The same locomotive may be TRAILING on one lap and LEADING on the next.

Signal-based release. Release conditions are signal-based, not timer-based. A timer may serve as a minimum floor — ensuring a locomotive does not depart before a minimum dwell has elapsed — but a timer alone is never sufficient to release a state transition that depends on another locomotive's position. Every release that involves inter-locomotive coordination must be gated on a received message, not on elapsed time.

This principle was established in v1.4 following field observation that TR_PLATFORM_DWELL was releasing the trailing locomotive on a timer alone. When the platformDwellMs value was long enough (10,000ms), the leader happened to reach the next station before the timer expired and the system appeared correct. When the timer was shortened, the failure became visible. The timer was not the right gate — the leader's arrival at the next station was. v1.4 replaces the timer-only gate with a signal gate.

How Role Is Determined

When a locomotive detects a block boundary (two-magnet Hall sensor pair), the CTO logic queries the peer observation table before stopping:

Q1 — Am I closing on a leader? Has another locomotive recently reported this same block and not yet moved beyond it? If yes, this locomotive is TRAILING. Q1 is evaluated first.

Q2 — Is someone closing on me? Has another locomotive recently reported the block immediately behind mine? If yes, this locomotive is LEADING. Only evaluated if Q1 is false.

Neither. No close-running relationship detected. Normal station stop (STATION profile).

Role Unknown. No block detected yet, system just started, or trains separated beyond the freshness window (45 seconds). DEFAULT profile applies.

The Four Stop Profiles

DEFAULT

Condition: stopReason == STOP_NONE

Intent: Safe neutral operation. No encounter logic. No waiting for any other locomotive. Applied at startup, when trains are separated, or whenever role cannot be determined. Does not include station hold behavior.

Departure: Immediate on dwell expiry. No conditions.

STATION

Condition: stopReason == STOP_STATION_DWELL

Intent: Normal scheduled stop. Q1 and Q2 both returned false. The locomotive waits for the configured dwell time. DEPARTING is broadcast on departure so any trailing loco waiting at this block is unblocked.

Departure: Immediate on dwell expiry. DEPARTING broadcast.

TRAILING

Condition: stopReason == STOP_CTO_TRAILING

Intent: Stop immediately behind the leader. Once confirmed stop is achieved, send ENCOUNTER_START and RELEASE to the leader. Then enter the trailing platform advance sequence — do NOT depart based on dwell alone.

Confirmed stop definition: PWM below 25 AND sustained for at least 250ms. RELEASE must not be sent before confirmed stop.

Trailing platform advance sequence (post-RELEASE):

TR_WAIT_LEADER_DEPART — Hold until MSG_TYPE_DEPARTING received from leader.

TR_FOLLOW_DELAY — Wait followDelayMs after DEPARTING (default 3000ms).

TR_ADVANCE_TO_PLATFORM — Ramp to advancePwm (default cruisePwm/2), run for advanceMs (default 10000ms).

TR_PLATFORM_STOP — Ramp down, wait for confirmed stop.

TR_PLATFORM_DWELL — Wait platformDwellMs (floor only — see signal gate requirement below). Then depart and broadcast DEPARTING.

TR_PLATFORM_DWELL signal gate (v1.4):

The trailing loco must not depart the platform on the dwell timer alone. Both conditions must be satisfied:

1. platformDwellMs has elapsed (minimum floor), AND
2. A MSG_TYPE_BLOCK has been received from the leader (trailingLeaderId) at a block different from the encounter block (trailingEncounterBlock), with departingReceived == true.

The second condition confirms the leader has physically reached the next station. Until both are true, the trailing loco remains stopped at the platform and logs a waiting message every 3 seconds.

Implementation: leaderReachedNextBlock flag, set in onReceive() on MSG_TYPE_BLOCK when the above conditions are met. Cleared in resetEncounterState(). TR_PLATFORM_DWELL checks this flag after timer expiry and returns if false.

Ramp delay: Zero — braking begins immediately on block detection.

LEADER

Condition: stopReason == STOP_CTO_LEADER_HOLD

Intent: Stop at the platform and hold. Wait for RELEASE from the trailing locomotive before departing. Dwell begins at confirmed physical stop and runs concurrently while waiting for RELEASE.

Ramp delay: 6000ms default — the leader coasts 6 seconds past the block detection point before braking, positioning it well into the platform area.

Departure gate — both conditions must be true:

Dwell time has expired, AND valid RELEASE received from the correct trailing locomotive with matching encounter ID. Neither condition alone is sufficient.

On departure: Send MSG_TYPE_DEPARTING. This is the gate signal that begins the trailing loco's platform advance sequence.

MSG_TYPE_DEPARTING — Critical Protocol

Added in v1.2. Sent by any loco at the exact moment rampTarget is set positive after a stop. This message gates the trailing loco's platform advance. Without it the trailing loco waits indefinitely.

Key rule: DEPARTING must be broadcast on every departure — not just LEADER departures. A STATION departure broadcast unblocks any trailing loco waiting at that block. Without this, deadlock occurs.

The DEPARTING packet carries the sender's locold, blockId, and encounterId. When sent from a STATION or DEFAULT departure, encounterId is 0 and target locold is 0 (broadcast). The trailing loco accepts any DEPARTING from its known leader at the matching block, regardless of encounter ID.

Confirmed Stop — Critical Definition

A locomotive is considered fully stopped when:

PWM (rampCurrent) is at or below 25, AND that condition has been sustained for at least 250ms continuously.

Sending RELEASE before physical stop allows the leader to depart while the trailing locomotive is still rolling — a rear-end collision risk. The debounce requirement is non-negotiable. If PWM rises above 25 during the debounce window, the timer resets.

Dwell Sequencing

Dwell starts at confirmed physical stop — not at block detection, not at RELEASE receipt.

Dwell runs concurrently — LEADER dwell runs while waiting for RELEASE, not after.

Dwell is a minimum floor, not a trigger — expiry opens the gate but does not force departure.

TRAILING dwell is minimal (500ms default) — departure is gated on DEPARTING from leader, not dwell.

TR_PLATFORM_DWELL is gated on both timer AND leader's next BLOCK signal (v1.4).

Ramp System

All PWM changes are managed exclusively by serviceRamp(). No other function calls ledcWrite() directly. This ensures smooth, predictable motion and prevents abrupt changes.

Deceleration (rampRateMs): Milliseconds between each PWM decrement during braking. Higher = slower, gentler braking. Lower = harder, shorter stop. Adjustable via rr####.

Acceleration (rampUpRateMs): Milliseconds between each PWM increment during acceleration. Default 120ms/step. Adjustable via ru####.

Ramp Delay (rampDelayMs): Milliseconds the loco coasts at cruise speed after block detection before braking begins. Leader: 6000ms default. Trailing: 0ms. Adjustable per role via Dispatcher.

Trailing Platform Advance Parameters

These parameters control the trailing loco's journey to the platform after RELEASE. All are runtime-adjustable from the Dispatcher without reflashing.

Parameter	Default	Command	Description
followDelayMs	3000ms	bfd3000	Wait after DEPARTING before advancing
advanceMs	10000ms	bad10000	Time spent moving toward platform
advancePwm	cruise/2	bap60	PWM during advance (override 0=auto)
platformDwellMs	10000ms	bpd10000	Minimum time stopped at platform (floor only — signal gate also required)

Complete Tuning Parameter Reference

Parameter	Scope	Dispatcher Command	Default
Cruise PWM	Per loco	op#### / tp#### / bp####	Config file
Ramp delay	Per role	[o t b][role]rd####	See profiles
Ramp rate (decel)	All profiles	rr####	80ms/step
Ramp up rate	All depart.	ru####	120ms/step
Dwell	Per role	[o t b][role]dw####	See profiles
Follow delay	Trailing	[o t b]fd#####	3000ms
Advance duration	Trailing	[o t b]ad#####	10000ms

Advance Pwm	Trailing	[o t b]ap###	cruisePwm/2
Platform dwell (floor)	Trailing	[o t b]pd#####	10000ms

Role abbreviations: df=DEFAULT st=STATION tr=TRAILING ld=LEADER. Target: o=Otto t=Toby b=both. All changes take effect immediately and reset to compiled defaults on power cycle.

Compiled Default Profile Values

Profile	Ramp Delay	Ramp Rate	Dwell	Notes
DEFAULT	200ms	80ms/step	2000ms	No encounter logic
STATION	200ms	80ms/step	5000ms	Normal stop, DEPARTING on exit
TRAILING	0ms	53ms/step	500ms	Immediate brake; departs via sub-states
LEADER	6000ms	80ms/step	4000ms	Coasts to platform; holds for RELEASE

rampUpRateMs = 120ms/step (all profiles). trailingAdvancePwm = cruisePwm/2 (auto).

ESP-NOW Message Protocol

All communication uses a fixed 28-byte packed struct broadcast to MAC FF:FF:FF:FF:FF:FF. Every node receives every packet. Nodes filter by magic (0x4E), version, and senderId. Packet VERSION must match across all three nodes — mismatches are silently dropped.

Message	Sender	Key recipient action
MSG_TYPE_BLOCK	Any loco	All nodes update peer table with block/timestamp. Trailing loco checks leaderReachedNextBlock
MSG_TYPE_ENCOUNTER_START	Trailing	Leader notes trailing has arrived at block
MSG_TYPE_RELEASE	Trailing	Leader sets leaderReleaseReceived=true
MSG_TYPE_DEPARTING	Any loco	Trailing sets departingReceived=true; begins follow delay
MSG_TYPE_ESTOP	Dispatcher	All locos: running=false, rampTarget=0, state=STOP_ESTOP
MSG_TYPE_MANUAL	Dispatcher	Locos act on embedded CmdType and parameters

Typical Two-Train Encounter Sequence

The behavioral result of a correctly functioning CTO encounter at a block:

Step	Event
1	Leader arrives at block, coasts 6s, stops at platform (LEADER profile)
2	Trailing arrives, brakes immediately, stops behind leader (TRAILING profile)
3	Trailing: confirmed stop → sends ENCOUNTER_START + RELEASE to leader
4	Leader: dwell expired + RELEASE received → sends DEPARTING → departs
5	Trailing: DEPARTING received → follow delay 3s
6	Trailing: advances to platform at cruisePwm/2 for 10s
7	Trailing: platform stop (confirmed) → platform dwell begins (10s floor)

8	Trailing: waits for leader's MSG_TYPE_BLOCK at next station (v1.4 signal gate)
9	Trailing: both conditions satisfied → departs, broadcasts DEPARTING → resumes normal running

Steps 8 and 9 are new in v1.4. Step 7 (platform dwell) previously triggered step 9 directly — timer only. The signal gate in step 8 is the v1.4 fix.

E-STOP

E-STOP is not a profile. It overrides all profile and encounter logic immediately. A locomotive receiving E-STOP sets running=false, rampTarget=0, stopReason=STOP_ESTOP and will not respond to CMD_START until power-cycled or manually reset. E-STOP is broadcast to all nodes simultaneously.

What Remains Locomotive-Specific

Cruise PWM — each locomotive may require a different cruise speed reflecting its motor characteristics, wheel condition, and weight. This is the only parameter that remains per-locomotive. All other tuning parameters are role-based and apply identically to both locos.

Deferred — Future Versions

- Role lock at block boundary — prevent role flipping near detection edge cases
- Selective encounter engagement — Dispatcher designates which blocks trigger full CTO sequence vs. pass-through
- Skip-stop behavior — trailing loco passes through a station without stopping to re-establish spacing after a gap opens
- Leader double-hop — leader skips the next station and continues to the one after; trailing does its normal routine and meets leader two stations ahead
- Randomized dwell — small variation per stop to prevent long-term synchronization
- Persistent parameter storage across power cycles (SPIFFS/NVS)
- Self-calibration of stopping distance based on measured overshoot
- Support for more than two locomotives
- Full Work Zone integration — currently stubbed in Dispatcher
- Unified Dispatcher for Lowline and Highline with MIT physical controller

Relationship to CTO Architecture

The stopReason enum is the profile selection hook — set at block detection before any profile values are accessed. Profiles are selected in startDwellAtBlock() and applied throughout serviceDwelling() and serviceRamp().

serviceRamp() is the sole owner of ledcWrite(). All other code changes rampTarget only. serviceRamp() moves rampCurrent toward rampTarget one step per interval, using rampRateMs for deceleration and rampUpRateMs for acceleration.

The trailing sub-state machine runs inside `serviceDwelling()` after confirmed stop and RELEASE have been sent. It is a separate sequential state machine that runs to completion before normal running resumes. The main `detectState` remains DWELLING throughout the entire trailing sequence.

The `leaderReachedNextBlock` flag is set in `onReceive()` on `MSG_TYPE_BLOCK` when `trailingEncounterActive`, the sender is `trailingLeaderId`, `departingReceived` is true, and `pkt.blockId != trailingEncounterBlock`. It is cleared in `resetEncounterState()`. This flag is the v1.4 signal gate.